

Aula 03 — Assincronia: quando o código precisa esperar

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

Duração: 2h · **Pré-requisito:** Aula 02 · **Precisa de internet:** sim (segunda metade)

Objetivos de aprendizagem

Ao final desta aula o aluno deve conseguir:

1. Explicar a diferença entre código síncrono e assíncrono e por que o Node é assíncrono.
2. Reconhecer uma Promise e entender seus três estados.
3. Escrever e ler código com `async / await`.
4. Tratar erros com `try / catch` em código assíncrono.
5. Consumir uma API externa com `fetch` e ler/escrever arquivos com `fs/promises`.

Plano de aula (2h)

Tempo	Bloco	Conteúdo
0:00–0:10	Aquecimento	Correção do exercício 5 da aula 02 (imutabilidade)
0:10–0:35	Conceito	Síncrono x assíncrono · o garçom · callbacks → Promises
0:35–1:05	Código ao vivo	<code>async/await</code> · <code>try/catch</code> · <code>fetch</code> numa API real
1:05–1:15	Intervalo	
1:15–1:50	Mão na massa	Exercícios 1 a 4
1:50–2:00	Fechamento	<code>Promise.all</code> · prévia da aula 04

1. O problema: algumas coisas demoram

Quase tudo em JavaScript é instantâneo:

```
const soma = 2 + 3;           // instantâneo
const maiusculo = "a".toUpperCase(); // instantâneo
```

Mas algumas operações dependem do **mundo lá fora** e levam tempo:

Operação	Tempo típico
Somar dois números	0,000001 ms
Ler um arquivo do disco	5 ms
Consultar um banco de dados	20 ms
Chamar uma API na internet	300 ms

300 ms parece pouco. Mas o JavaScript executa **uma coisa de cada vez**. Se o programa parasse por 300 ms a cada requisição, um servidor atenderia no máximo 3 pessoas por segundo. Inaceitável.

A analogia do garçom

Imagine um garçom em um restaurante:

- **Modo síncrono (bloqueante):** ele anota seu pedido, vai para a cozinha, fica parado olhando o fogão até o prato ficar pronto, traz o prato, e só então atende a próxima mesa. Um garçom, uma mesa por vez. 🐌
- **Modo assíncrono:** ele anota seu pedido, entrega na cozinha, e **vai atender outras mesas**. Quando a cozinha grita "pedido 5 pronto!", ele busca e entrega. Um garçom, dez mesas ao mesmo tempo. 🚀

O Node.js é o segundo garçom. Ele não fica parado esperando. É exatamente por isso que ele aguenta milhares de usuários simultâneos com um único processo.

Importante: o Node **não faz duas coisas ao mesmo tempo**. Ele apenas nunca fica parado esperando. Enquanto o disco ou a rede trabalham, ele executa outro pedaço de código.

2. O jeito antigo: callbacks

A primeira solução do JavaScript foi: *"me dê uma função e eu a chamo quando terminar"*.

```
setTimeout(() => {  
  console.log("passaram 2 segundos");  
}, 2000);  
  
console.log("esta linha aparece PRIMEIRO");
```

Note a ordem da saída. A segunda linha não espera a primeira. **Essa é a assincronia acontecendo.**

O problema aparece quando uma coisa depende da outra:

```

buscarUsuario(1, (usuario) => {
  buscarPedidos(usuario.id, (pedidos) => {
    buscarItens(pedidos[0].id, (itens) => {
      calcularFrete(itens, (frete) => {
        console.log(frete); // 🤔 e ainda faltam os erros de cada nível
      });
    });
  });
});

```

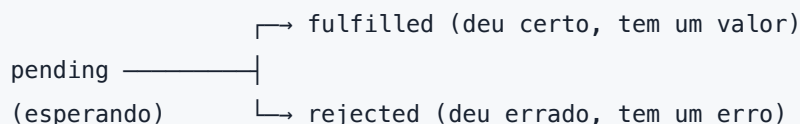
Isso tem nome: **callback hell**, ou "pirâmide da perdição". O código cresce para a direita e vira um pesadelo de ler e de tratar erros.

3. Promise: um vale para um valor futuro

Uma **Promise** é um objeto que representa um valor que **ainda não chegou**.

Analogia: você pede um lanche e recebe uma senha. A senha não é o lanche — é a **promessa** de que ele virá. Você guarda a senha no bolso e continua sua vida. Quando chamarem, você troca a senha pelo lanche. Ou pela notícia de que o pão acabou.

Toda Promise está em um de três estados:



Uma vez resolvida, ela **nunca mais muda de estado**.

```

const promessa = fetch("https://viacep.com.br/ws/01001000/json/");
console.log(promessa); // Promise { <pending> } ← ainda não chegou!

```

Consumindo com `.then()`

```

fetch("https://viacep.com.br/ws/01001000/json/")
  .then((resposta) => resposta.json())
  .then((dados) => console.log(dados.logradouro))
  .catch((erro) => console.log("deu ruim:", erro));

```

Melhor que callbacks — o código encadeia em vez de aninhar. Mas ainda não é o ideal.

4. `async` / `await`: o jeito moderno

`async/await` é uma forma de escrever Promises que **parece código normal, de cima para baixo**.

```
async function buscarCep(cep) {  
  const resposta = await fetch(`https://viacep.com.br/ws/${cep}/json/`);  
  const dados = await resposta.json();  
  return dados;  
}
```

Duas regras, só isso:

1. `await` = "espere esta Promise resolver e me dê o valor de dentro".
2. `async` na função = "esta função contém `await` e sempre devolve uma Promise".

Em arquivos ESM (nosso caso), você pode usar `await` no topo do arquivo, sem estar dentro de uma função `async`. Chama-se *top-level await*. Dentro de funções, o `async` é obrigatório.

⚠ O erro nº 1 do módulo: esquecer o `await`

```
const dados = fetch(url);           // ❌ Promise { <pending> }  
const dados = await fetch(url);     // ✅ o valor de verdade
```

Se em algum momento você ver `Promise { <pending> }` no `console.log`, **faltou um `await`**. Grave isso. Vai te economizar horas nas próximas semanas.

Toda função `async` devolve uma Promise

```
async function pegarNumero() {  
  return 42;           // repare: devolvo um número...  
}  
  
console.log(pegarNumero());           // Promise { 42 } ← ...mas sai uma Promise  
console.log(await pegarNumero());    // 42
```

Isso não é detalhe: significa que **quem chama uma função `async` também precisa dar `await`**. O `async` "contamina" a cadeia de chamadas — e isso é normal.

5. Tratando erros

Em código assíncrono, um erro que ninguém trata derruba o programa inteiro. Em um servidor, isso significa que **todos os usuários caem**. Tratar erro não é capricho.

```
async function buscarCep(cep) {
  try {
    const resposta = await fetch(`https://viacep.com.br/ws/${cep}/json/`);

    // ⚠️ ATENÇÃO: o fetch NÃO lança erro em status 404 ou 500.
    // Ele só falha se a rede cair. Você precisa checar na mão:
    if (!resposta.ok) {
      throw new Error(`A API respondeu com status ${resposta.status}`);
    }

    return await resposta.json();
  } catch (erro) {
    console.error("Falha ao buscar o CEP:", erro.message);
    return null; // devolvemos algo previsível em vez de quebrar
  } finally {
    console.log("busca encerrada"); // roda sempre, com erro ou sem
  }
}
```

Guarde essas três:

- `try` — o código que **pode** dar errado.
- `catch (erro)` — o que fazer **se** der. O objeto `erro` tem `erro.message`.
- `finally` — roda **sempre**, deu certo ou não. Bom para limpeza.

6. Esperando várias coisas ao mesmo tempo

Situação real: buscar 3 CEPs.

```
// ❌ SEQUENCIAL: 300ms + 300ms + 300ms = 900ms
const a = await buscarCep("01001000");
const b = await buscarCep("20040030");
const c = await buscarCep("30130010");
```

Cada `await` espera o anterior terminar. Mas eles são independentes — não precisam de fila!

```
// ✅ PARALELO: as três saem juntas = ~300ms
const [a, b, c] = await Promise.all([
  buscarCep("01001000"),
  buscarCep("20040030"),
  buscarCep("30130010"),
]);
```

Regra prática: se uma chamada precisa do resultado da anterior, use `await` em sequência. Se são independentes, use `Promise.all`.

⚠️ `Promise.all` é "tudo ou nada": se **uma** falhar, todas são descartadas. Quando você quer o resultado de cada uma, mesmo com falhas, use `Promise.allSettled`.

A armadilha do `forEach`

```
// ❌ NÃO funciona – o forEach não espera nada
ceps.forEach(async (cep) => {
  const dados = await buscarCep(cep);
  console.log(dados);
});
console.log("acabou"); // imprime ANTES de qualquer dado chegar

// ✅ funciona
for (const cep of ceps) {
  const dados = await buscarCep(cep);
  console.log(dados);
}
```

O `forEach` ignora completamente a Promise que o callback devolve. Use `for...of` (sequencial) ou `Promise.all` com `map` (paralelo).

7. Lendo e escrevendo arquivos

O Node consegue mexer em arquivos — algo que o navegador nunca pôde. A versão moderna do módulo `fs` já usa Promises:

```
import { readFile, writeFile } from "node:fs/promises";

// ler (o "utf-8" é o que faz voltar texto em vez de bytes crus)
const conteudo = await readFile("dados.txt", "utf-8");

// escrever (substitui o arquivo inteiro)
await writeFile("saida.txt", "olá");

// acrescentar ao final
import { appendFile } from "node:fs/promises";
await appendFile("log.txt", "nova linha\n");
```

O prefixo `node:` deixa explícito que o módulo é interno do Node, não algo baixado do npm. É a forma recomendada hoje.

Para trabalhar com JSON em arquivo, o par é sempre este:

```
const texto = await readFile("dados.json", "utf-8");
const dados = JSON.parse(texto); // texto → objeto

await writeFile("dados.json", JSON.stringify(dados, null, 2)); // objeto → texto
//                                     ↑      ↑
//                                     (sem   indentação de
//                                     filtro) 2 espaços, legível
```

8. Ferramentas novas desta aula

`setTimeout(funcao, milissegundos)`

O que faz: executa a função depois do tempo indicado. **Quando usar:** adiar algo, simular demora, ou esperar o usuário parar de digitar (faremos isso na aula 09).

```
setTimeout(() => console.log("oi"), 1000); // 1 segundo = 1000 ms

const timer = setTimeout(fn, 400);
clearTimeout(timer); // cancela antes de disparar
```

`fetch(url, opcoes)`

O que faz: faz uma requisição HTTP. **Devolve:** uma Promise com a **resposta** — não com os dados. São sempre dois passos:

```
const resposta = await fetch(url);           // 1. a resposta
const dados = await resposta.json();         // 2. os dados – TAMBÉM é assíncrono
```

Propriedade da resposta	O que é
<code>resposta.ok</code>	<code>true</code> se o status for 2xx
<code>resposta.status</code>	o número: 200, 404, 500...
<code>resposta.json()</code>	converte o corpo em objeto (assíncrono)
<code>resposta.text()</code>	o corpo como texto puro

`throw` e o objeto de erro

`throw new Error("mensagem")` interrompe a execução e pula direto para o `catch` mais próximo. Lá, você recebe o objeto de erro:

Propriedade	O que tem	Use para
<code>erro.message</code>	a mensagem legível	mostrar ao usuário
<code>erro.name</code>	o tipo do erro	decidir o tratamento
<code>erro.stack</code>	onde aconteceu	só para o log; nunca mostre ao usuário
<code>erro.code</code>	código do sistema (<code>"ENOENT"</code> , por exemplo)	tratar casos específicos

`Promise.all([...])`

O que faz: espera **todas** as promises da lista e devolve um array com os resultados, na mesma ordem em que foram passadas.

```
const [a, b] = await Promise.all([buscar("x"), buscar("y")]);
```

⚠ É "tudo ou nada": se **uma** falhar, você perde o resultado das outras. Quando quiser o resultado de cada uma, mesmo com falhas, use `Promise.allSettled`.

`new Promise((resolve, reject) => {...})`

Criar promise na mão é raro — as bibliotecas já devolvem promises prontas. A exceção que aparece sempre é o "esperar":

```
function esperar(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

await esperar(1000);
```

- `resolve(valor)` → deu certo; esse valor é o que o `await` devolve.
- `reject(erro)` → deu errado; esse erro cai no `catch`.

`performance.now()`

O que faz: devolve um número de milissegundos, usado para medir quanto tempo algo levou.

```
const inicio = performance.now();
// ... algo demorado ...
console.log(`${Math.round(performance.now() - inicio)} ms`);
```

Alternativa mais simples: `console.time("nome")` e `console.timeEnd("nome")`.

`readFile`, `writeFile` e `appendFile`

Função	O que faz
<code>readFile(arquivo, "utf-8")</code>	lê o conteúdo
<code>writeFile(arquivo, texto)</code>	substitui o arquivo inteiro
<code>appendFile(arquivo, texto)</code>	acrescenta ao final

⚠ Dois detalhes que pegam todo mundo:

```
await readFile("dados.txt"); // Buffer <48 65 6c...> - bytes crus 😞
await readFile("dados.txt", "utf-8"); // "Olá mundo" ✅
```

E `writeFile` **substitui tudo**. Se você quer só acrescentar uma linha, é `appendFile`.

`erro.code === "ENOENT"`

Erros do sistema de arquivos trazem um `code`. O mais comum é `ENOENT` — *Error NO ENTry*, ou seja, "esse arquivo não existe".

```
try {  
  return JSON.parse(await readFile(ARQUIVO, "utf-8"));  
} catch (erro) {  
  if (erro.code === "ENOENT") return []; // primeira execução: esperado  
  throw erro; // qualquer outro: não é esperado, repassa  
}
```

Repare no padrão: **trate o erro que você espera, repasse o que você não espera**. Engolir todos os erros com um `catch` vazio é uma das piores práticas que existem.

`.slice()` em texto

```
"2026-04-01T10:00:00.000Z".slice(0, 10); // "2026-04-01"
```

Recorta um pedaço do texto. O segundo número (o fim) **não** entra no recorte.

Recapitulação em 3 frases

1. O Node não fica parado esperando: ele delega e continua — por isso aguenta muita gente ao mesmo tempo.
2. `await` desembulha uma Promise; sem ele, você recebe `Promise { <pending> }`.
3. Independentes vão juntas com `Promise.all`; sempre embrulhe o que pode falhar em `try/catch`.

Tarefa para a próxima aula

Exercícios 1 a 4 de [exercicios/README.md](#).

Na próxima aula

Chegou a hora: vamos **criar um servidor**. Você vai entender o que acontece entre digitar um endereço no navegador e a página aparecer — e escrever o programa que fica do outro lado.